

机器学习基础：神经网络优化与参数初始化

1 实验目的

本次上机练习要求同学们基于一个图像分类任务，系统比较神经网络训练过程中若干关键因素对模型性能的影响，包括：

1. 比较 MSE 与交叉熵损失对分类任务训练效果的影响；
2. 测试不同 batch size 与学习率 pair 对模型训练的影响；
3. 测试不同优化算法、学习率衰减策略对模型训练的影响；
4. 测试不同参数初始化方案对模型训练的影响，包括全 0 初始化、Xavier 初始化、He 初始化、uniform 初始化和 normal 初始化。

通过本实验，同学们应理解：神经网络训练效果不仅由模型结构决定，损失函数、优化算法、学习率、batch size 和初始化方法都会显著影响模型的收敛速度、训练稳定性与泛化性能。

2 实验数据与基础模型

2.1 数据集

建议使用 MNIST 或 Fashion-MNIST 数据集完成实验。每个样本为一张灰度图像，类别数为 10。训练目标是完成多分类任务。

数据加载示例：

```
import tensorflow as tf

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data
()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

2.2 基础模型

统一使用如下简单前馈神经网络作为基准模型：

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

除当前实验变量外，其他设置应尽量保持一致。例如固定训练轮数、数据划分方式、随机种子等。建议统一训练：

```
epochs = 10
```

每组实验至少记录以下指标：

```
train_loss
val_loss
train_accuracy
val_accuracy
test_accuracy
训练时间
```

并绘制训练曲线。

3 实验一：比较 MSE 与交叉熵损失对分类任务的影响

3.1 实验内容

在相同模型、相同优化器、相同 batch size 和相同学习率下，分别使用 MSE 与交叉熵损失训练模型，并比较二者在分类任务中的表现。

交叉熵损失用于分类问题，可以直接衡量真实类别分布与预测概率分布之间的差异。MSE 常用于回归问题，也可以用于分类实验中的对比，但通常不是分类任务的首选损失函数。

使用 MSE 时，由于模型输出是 10 维 softmax 概率向量，需要将标签转换为 one-hot 编码：

```
y_train_onehot = tf.keras.utils.to_categorical(y_train, num_classes=10)
y_test_onehot = tf.keras.utils.to_categorical(y_test, num_classes=10)
```

交叉熵损失可以直接使用整数标签：

```
loss = 'sparse_categorical_crossentropy'
```

3.2 推荐实验设置

表 1: MSE 与交叉熵损失比较实验

实验编号	损失函数	优化器	学习率	Batch size
A1	MSE	SGD	0.01	128
A2	Sparse Categorical Crossentropy	SGD	0.01	128

3.3 需要回答的问题

1. 哪一种损失函数下降更快？
2. 哪一种损失函数最终测试精度更高？
3. 为什么分类任务通常更适合使用交叉熵损失，而不是 MSE？
4. 当模型输出是 softmax 概率时，交叉熵损失如何更直接地惩罚错误类别的高置信度预测？

4 实验二：测试不同 batch size 和学习率 pair 对训练的影响

4.1 实验内容

固定模型结构、损失函数和优化器，改变 batch size 与 learning rate 的组合，观察训练曲线和测试精度变化。

统一使用：

```
loss = 'sparse_categorical_crossentropy'  
optimizer = tf.keras.optimizers.SGD(learning_rate=lr)
```

4.2 推荐实验组合

4.3 观察重点

比较不同组合下的训练现象：

```
loss 是否稳定下降  
accuracy 是否稳定提升  
是否出现震荡
```

表 2: Batch size 与学习率 pair 比较实验

实验编号	Batch size	Learning rate
B1	32	0.001
B2	32	0.01
B3	32	0.1
B4	128	0.001
B5	128	0.01
B6	128	0.1
B7	512	0.001
B8	512	0.01
B9	512	0.1

是否出现不收敛
是否训练速度过慢

4.4 需要回答的问题

1. 当 batch size 较小时，学习率过大会发生什么？
2. 当 batch size 较大时，为什么通常可以尝试较大的学习率？
3. batch size 变大后，每个 epoch 的训练曲线是否更平滑？
4. 你认为本实验中最合适的 batch size 和 learning rate 组合是哪一组？请说明理由。

5 实验三：测试不同优化算法与学习率衰减策略的影响

5.1 实验内容

固定模型结构、损失函数和 batch size，比较不同优化算法和学习率策略对训练效果的影响。

损失函数统一使用：

```
loss = 'sparse_categorical_crossentropy'
```

batch size 建议固定为：

```
batch_size = 128
```

5.2 优化算法比较

表 3: 优化算法比较实验

实验编号	优化器	学习率
C1	SGD	0.01
C2	SGD + Momentum	0.01
C3	SGD + Nesterov	0.01
C4	RMSprop	0.001
C5	Adam	0.001
C6	Adagrad	0.01

示例代码如下：

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
```

```
optimizer = tf.keras.optimizers.SGD(  
    learning_rate=0.01,  
    momentum=0.9  
)
```

```
optimizer = tf.keras.optimizers.SGD(  
    learning_rate=0.01,  
    momentum=0.9,  
    nesterov=True  
)
```

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
```

5.3 学习率衰减策略比较

在 SGD 或 Adam 下，进一步测试不同学习率衰减策略。

5.3.1 固定学习率

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
```

5.3.2 指数衰减

```
lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(  
    initial_learning_rate=0.01,  
    decay_steps=1000,  
    decay_rate=0.9  
)  
  
optimizer = tf.keras.optimizers.SGD(learning_rate=lr_schedule)
```

5.3.3 逆时衰减

```
lr_schedule = tf.keras.optimizers.schedules.InverseTimeDecay(  
    initial_learning_rate=0.01,  
    decay_steps=1000,  
    decay_rate=0.5  
)  
  
optimizer = tf.keras.optimizers.SGD(learning_rate=lr_schedule)
```

5.4 推荐实验组合

表 4: 优化算法与学习率衰减策略组合实验

实验编号	优化器	学习率策略
D1	SGD	固定学习率
D2	SGD	ExponentialDecay
D3	SGD	InverseTimeDecay
D4	SGD + Momentum	固定学习率
D5	SGD + Momentum	ExponentialDecay
D6	Adam	固定学习率
D7	Adam	ExponentialDecay

5.5 需要回答的问题

1. SGD、Momentum、Nesterov、Adam 中，哪一种收敛最快？
2. Momentum 是否减少了 loss 曲线的震荡？
3. Adam 是否在前几个 epoch 中具有更快的下降速度？

4. 学习率衰减是否改善了后期训练的稳定性？
5. 固定学习率和衰减学习率相比，哪一种最终验证集精度更高？

6 实验四：不同参数初始化方案对模型训练的影响

6.1 实验内容

固定模型结构、损失函数、优化器、batch size 和 learning rate，仅改变 Dense 层的参数初始化方式，比较训练效果。

统一使用：

```
loss = 'sparse_categorical_crossentropy'
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
batch_size = 128
epochs = 10
```

6.2 需要测试的初始化方案

表 5: 参数初始化方案比较实验

实验编号	初始化方法	Keras 写法
E1	全 0 初始化	<code>kernel_initializer='zeros'</code>
E2	Xavier uniform	<code>kernel_initializer='glorot_uniform'</code>
E3	Xavier normal	<code>kernel_initializer='glorot_normal'</code>
E4	He uniform	<code>kernel_initializer='he_uniform'</code>
E5	He normal	<code>kernel_initializer='he_normal'</code>
E6	Uniform	<code>RandomUniform(...)</code>
E7	Normal	<code>RandomNormal(...)</code>

示例模型函数：

```
def build_model(initializer):
    model = tf.keras.Sequential([
        tf.keras.layers.Flatten(input_shape=(28, 28)),
        tf.keras.layers.Dense(
            256,
            activation='relu',
            kernel_initializer=initializer,
            bias_initializer='zeros'
```

```
),
tf.keras.layers.Dense(
    128,
    activation='relu',
    kernel_initializer=initializer,
    bias_initializer='zeros'
),
tf.keras.layers.Dense(
    10,
    activation='softmax',
    kernel_initializer=initializer,
    bias_initializer='zeros'
)
])
return model
```

6.3 全 0 初始化实验

先测试：

```
initializer = 'zeros'
```

观察模型是否能够正常训练，并重点回答：

1. 全 0 初始化为什么不行？
2. 全 0 初始化会导致隐藏层神经元学到相同的特征吗？
3. 为什么 bias 可以初始化为 0，但 weight 不应该全部初始化为 0？

参考分析方向：全 0 初始化会导致同一层中的神经元具有完全相同的初始权重，因此它们接收到相同输入，产生相同输出，并在反向传播中得到相同梯度。这样每个神经元的更新方向完全一致，网络无法打破对称性，相当于多个神经元在重复学习同一个特征，模型表达能力大幅下降。

6.4 Xavier 与 He 初始化比较

由于本实验使用 ReLU 激活函数，理论上 He 初始化通常更适合 ReLU 网络。需要比较：

```
glorot_uniform
glorot_normal
he_uniform
he_normal
```


观察：

初始 **loss** 是否较低
前几个 **epoch** 是否下降更快
最终测试精度是否更高
训练曲线是否更稳定

6.5 Uniform 与 Normal 初始化比较

可以自定义如下初始化：

```
tf.keras.initializers.RandomUniform(minval=-0.05, maxval=0.05)
```

```
tf.keras.initializers.RandomNormal(mean=0.0, stddev=0.05)
```

进一步尝试较大方差：

```
tf.keras.initializers.RandomNormal(mean=0.0, stddev=1.0)
```

观察是否出现训练不稳定、loss 震荡或精度下降。